

ARM Cortex M4 Architecture

EMBEDDED — Lecture 02

Sections: The Processor as the Software Interface; Architecture vs Microarchitecture; CPU Architecture Models; Cortex-M4 Overview; Cortex-M4 Block Diagram; Programming Model and Registers; Cortex-M4 Memory Map; Bit-band Operations; Endianness; Arm and Thumb Instruction Set.

The Processor as the Software Interface

- The hardware architecture forms the **interface for the embedded software**; one must understand the basic features offered by the processor inside the microcontroller. The lecture is based on Arm University material.
- Embedded software sees **registers, memory regions, exceptions, and instructions**. The processor architecture defines the **rules that code must follow**.

Arm Architecture and Ecosystem

A 32-bit Arm Cortex-M4 MCU family includes platforms such as TI MSP432 and STM32 Nucleo F4: **different vendors, one common Cortex-M programming model**.

Architecture	Defines the instruction set, registers, exception behavior, and memory model.
Processor core	Implements an Arm architecture as a concrete microarchitecture, such as Cortex-M4.
Microcontroller	Combines the core with memory, clocks, buses, and peripherals from a semiconductor vendor.

RISC and CISC

CISC (complex instruction set computer)	RISC (reduced instruction set computer)
- a variety of instructions that perform very complex tasks; e.g. string searching. - a number of different instruction formats of varying lengths.	- fewer and simpler instructions. - generally use load/store instruction sets: operate only on registers, cannot operate directly on memory locations. - instructions can be efficiently executed in a pipelined manner.

Arm Architectures and Processors

- Arm (Advanced RISC Machines) architecture** is a family of **RISC-based processor architectures**.
 - well known for its **power efficiency**;
 - widely used in **mobile devices**, e.g. smartphones and tablets;
 - designed and licensed by Arm to a wide ecosystem of partners.

The Arm Business Model

- Arm **licenses designs instead of manufacturing chips**: companies add their own intellectual property (IP) on top of Arm's IP, which they then fabricate and sell to customers.
- An **architectural license** allows a company to build its own compatible microarchitecture.

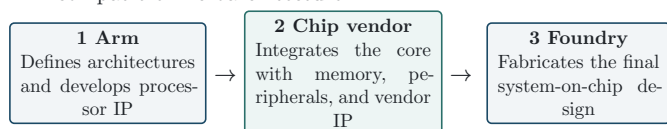


Fig. 1: From architecture to silicon: Arm defines architectures and develops processor IP, the chip vendor integrates the core with memory, peripherals, and vendor IP, and the foundry fabricates the final system-on-chip design.

Arm-based System-on-Chip

- Choose **processor, memory, bus, and peripheral IP blocks**.
- Integrate them into **one coherent memory and interrupt system**.
- Verify** the design before semiconductor fabrication.

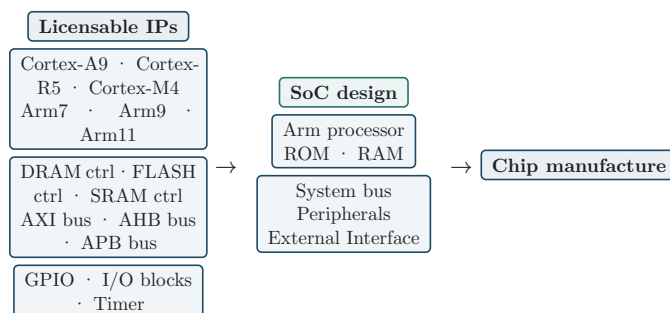


Fig. 2: Arm-based SoC combines reusable licensing IP: licensable IPs are assembled into an SoC design and then manufactured as a chip.

Cortex Families

Family	Target	Description and applications
Cortex-A	Application processors	high performance processors capable of full operating system (OS) support; applications include smartphones, digital TV, smart books
Cortex-R	Real-time processors	high performance and reliability for real-time applications; applications include automotive braking systems, powertrains
Cortex-M	Microcontroller processors	cost sensitive solutions for deterministic microcontroller applications; applications include microcontrollers, smart sensors

This course focuses on **Cortex-M**, which targets IoT devices.

Arm Cortex-M Series

- Energy-efficiency**: low energy cost supports longer battery life.
- Smaller code**: reduces flash and memory cost.
- Lower silicon costs**: less physical space.
- Ease of use and development**: shorten development cycles and promote reuse (lots of tools).
- A **broad vendor ecosystem** offers many peripheral combinations.

Cortex-M0 M0+	/	For applications requiring minimal cost, power and area; optimized for simple sensing and controlling.
Cortex-M3 M4 / M7	/	Designed for data intensive applications requiring higher performance, e.g. digital signal control. Cortex-M4 and Cortex-M7 integrate Digital Signal Processing (DSP) and accelerated floating point processing capability for fast and power-efficient algorithm processing.

Choosing a core Choosing a processor core is only one decision. Memory, peripherals, package, and power modes often dominate the final MCU choice.

Architecture vs Microarchitecture

- Architecture** defines those characteristics that must be true of all implementations.
- The **processor is the implementation**: different processors can execute the same architecture with different performance and energy behavior.

Architecture (ISA) — what software can rely on	Microarchitecture — how a processor implements it
Instructions; registers; memory model; exceptions	Pipeline depth; bus widths; caches; clock frequency

Processor	Arm tecture	Architec- ture	Core ture	Architec- ture	Hardware Multi- ply	Hardware Di- vide	DSP sions	Exten- sions	Floating Point
Cortex-M0	Armv6-M		Von Neumann		1 or 32 cycle	No	No		No
Cortex-M0+	Armv6-M		Von Neumann		1 or 32 cycle	No	No		No
Cortex-M3	Armv7-M		Harvard		1 cycle	Yes	No		No
Cortex-M4	Armv7E-M		Harvard		1 cycle	Yes	Yes		Optional
Cortex-M7	Armv7E-M		Harvard		1 cycle	Yes	Yes		Optional

Fig. 3: Arm Cortex-M series family: architecture, core architecture, arithmetic hardware, and DSP / floating-point extensions.

Arm Architecture Generations

Over time the Arm architecture has evolved, adding new architectural features that meet the growing demand for new functionality, integrated security features, high performance and the needs of new and emerging markets. An Arm processor is developed by considering one of the Arm architectures.

Arm architecture	Example processors
Armv4 / v4T	Arm7TDMI
Armv5 / v4E	Arm9926EJ-S
Armv6	Arm1136
Armv6-M	Cortex-M0, Cortex-M1
Armv7-A	Cortex-A9
Armv7-R	Cortex-R4
Armv7-M	Cortex-M4
Armv8-A	Cortex-A53, Cortex-A57
Armv8-R	
Armv8-M	

DA VERIFICARE The extracted label Arm9926EJ-S is reported identically by both text extractors but does not match the usual Arm naming; it may be ARM926EJ-S.

Cortex-M Family Table

CPU Architecture Models

Von Neumann Architecture

- **Memory holds both data and instructions:** the CPU fetches an instruction from memory, decodes it, and executes it.
- **CPU registers** store values used internally: program counter (PC), instruction register (IR), general-purpose registers, etc.



Fig. 4: Von Neumann architecture: a single memory holds data and instructions, connected to the CPU.

Harvard Architecture

- **Separate memories for data and program**, with separate instruction and data bus interfaces: this allows **two simultaneous memory fetches**.
- The program counter (PC) **points to program memory**, not **data memory**: it cannot use self-modifying code.
- Most DSPs are Harvard architectures, which provides higher bandwidth/performance.

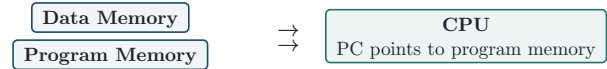


Fig. 5: Harvard architecture: separate data and program memories, each with its own bus to the CPU.

Cortex-M4 Overview

Signal Processing and Control

- Designed with a large variety of **highly efficient signal processing features** to accelerate filters and control loops, e.g. single-cycle multiply accumulate (MAC) instructions; optional floating-point support reduces software overhead.
- **Low power consumption:** longer battery life, especially critical in mobile products; low-power states reduce energy while the system waits.
- **Enhanced determinism:** fast exception handling improves response to asynchronous events; critical tasks and interrupt routines can be served quickly in a known number of cycles.

Processor Features

32-bit RISC core	Load-store RISC model with Harvard architecture.
Three-stage pipeline	Fetch, decode, and execute, with branch speculation.
Low power	Sleep modes with wait for interrupt (WFI) and wait for event (WFE) instructions and interrupt-based wake-up.
DSP support	Hardware multiply, MAC, saturation, and optional FPU.
Debug and trace	Breakpoints, watchpoints, and optional execution trace.

Core Subsystems

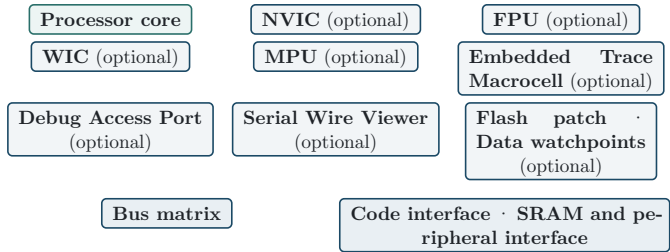


Fig. 6: Cortex-M4 core subsystems: the processor core is surrounded by optional modules, the bus matrix, and the code and SRAM/peripheral interfaces.

Cortex-M4 Block Diagram

Pipeline

- The **processor core** contains internal registers, the ALU, data path, and some control logic.
- The **three-stage pipeline** is fetch, decode, and execution; some instructions may take multiple cycles to execute; it speculatively prefetches instructions from branch target addresses.

Instruction	Fetch	Decode	Execute
Instruction 1	Fetch	Decode	Execute
Instruction 2	Fetch	Decode	Execute
Instruction 3	Fetch	Decode	Execute
Instruction 4	Fetch	Decode	Execute

Interrupt, Wake-up and Memory Protection

NVIC	Automatically handles nested interrupts, such as comparing priorities between interrupt requests and the current priority level.
nested vectored interrupt controller	
WIC	For low-power applications the microcontroller can enter sleep mode by shutting down most of the components; when an interrupt request is detected, the WIC can inform the power management unit to power up the system.
wake-up interrupt controller	
MPU	Used to protect memory content: make some memory regions read-only or prevent user applications from accessing.
memory protection unit	

Debug and Trace

- **Embedded Trace Macrocell (ETM)** can record instruction execution when implemented.
- **Data Watchpoint and Trace (DWT)** can observe selected data accesses.
- **Serial Wire Viewer (SWV)** streams selected trace data over a low-pin interface.
- **Debug subsystem:** handles debug control, program breakpoints, and data watchpoints. When a debug event occurs, it can put the processor core in a **halted state**, so developers can analyse the status of the processor, such as register values and flags, at that point.

Bus Interconnect

- The bus interconnect provides **data transfer management** among hardware components and peripherals, and allows data transfer to take place on **different buses simultaneously**.
 - **Advanced High-performance Bus (AHB)-Lite:** high bandwidth peripherals.
 - **Advanced Peripheral Bus (APB):** low-power, for peripherals such as timers, interrupt controllers, UARTs, and I/O ports.
- It may include **bus bridges** (e.g. AHB-to-APB bus bridge) to connect different buses into a network using a single global memory space.

Programming Model and Registers

Processor Registers

- The set of registers available for use by programs is called the **programming model**, also known as the **programmer model**. The CPU has many other registers used for internal operations that are unavailable to programmers.
- Processor registers store and process temporary data within the processor core quickly. Cortex-M4 uses a **load-store architecture**: to process memory data, it must first be loaded from memory to registers, processed inside the processor core using register data only, and then written back to memory if needed.
- Cortex-M4 has a **register bank** of 16 x 32-bit registers (13 are used for general-purpose) plus special registers.

Register Bank

R0–R12	General-purpose registers. Low registers (R0–R7) can be accessed by any instruction; high registers (R8–R12) sometimes cannot be accessed by some instructions.
R13	Stack Pointer (SP): records the current address of the stack; used for saving the context of a program while switching between tasks. Banked as Main SP (MSP) and Process SP (PSP).
R14	Link Register (LR): stores the return address of a subroutine or a function call; the PC loads the value from the LR after a function is finished.
R15	Program Counter (PC): records the address of the current instruction code; automatically incremented by four at each operation (for 32-bit instruction code), except branching operations.
xPSR	Program Status Register, combining APSR, EPSR, and IPSR.
PRIMASK • FAULT-MASK • BASEPRI • CONTROL	Interrupt mask and stack definition special registers.

The compiler assigns values to registers according to the calling convention and optimization strategy.

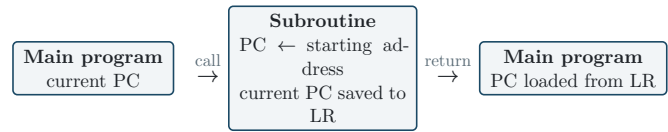


Fig. 7: Subroutine call and return: the current PC is saved to LR and the PC is loaded with the subroutine address; on return the PC is loaded from LR to resume the main program.

Special Registers: xPSR

The combined program status register **xPSR** provides information about program execution and ALU flags; it combines:

- **APSR** — Application PSR,
- **IPSR** — Interrupt PSR (ISR number),
- **EPSR** — Execution PSR (ICI/IT, T).

Register	Fields
APSR	N Z C V Q, then reserved
IPSR	reserved, ISR number
EPSR	ICI/IT, T, reserved, ICI/IT
xPSR	N Z C V Q, ICI/IT, T, reserved, ICI/IT, ISR number (from bit 31 down to bit 0)

APSR Flags and Saturation

N	negative flag
Z	zero flag
C	carry flag
V	overflow flag
Q	sticky saturation flag

Saturating arithmetic example: normal arithmetic would make 255 + 1 result in 0 (with overflow); saturating arithmetic makes 255 + 1 result in 255, as the result is clamped to the maximum value. Used in audio and video processing to ensure that signal levels stay within their defined bounds, preventing glitches and maintaining signal integrity.

Cortex-M4 Memory Map

Regions

- The memory map describes the organization of the processor’s address space.
- Arm memory is **byte-addressable using 32-bit addresses**; the Cortex-M4 processor has **4 GB** of memory address space.
- The 4 GB space is architecturally defined with a number of **regions**, each designed for particular recommended uses; this makes it easy for a software programmer to port between different devices.
- The memory map can also be **flexibly defined by the user**, apart from some fixed memory addresses such as the internal private peripheral bus.

Region	Base	Size	Typical use
Vendor specific Memory	0xE0100000	(reserved)	Reserved for other purposes
Private Peripheral Bus (PPB)	0xE0000000	512 MB	Private peripherals, e.g. NVIC, SCS
External device	0xA0000000	1 GB	External peripherals, e.g. SD card
External RAM	0x60000000	1 GB	External memories, e.g. DDR, FLASH, LCD
Peripherals	0x40000000	512 MB	On-chip peripherals, e.g. AHB, APB peripherals
SRAM	0x20000000	512 MB	Data memory, e.g. on-chip SRAM, SDRAM
Code	0x00000000	512 MB	Program code, e.g. on-chip FLASH

- **Code region:** primarily used to store program code; can also be used for data memory, such as on-chip FLASH.
- **SRAM region:** primarily used to store data, such as heaps and stacks; can also be used for program code.
- **Peripheral region:** primarily used for peripherals, such as Advanced High-performance Bus (AHB) or Advanced Peripheral Bus (APB) peripherals; on-chip peripherals.
- **PPB internals:** ROM table; External PPB (Embedded Trace Macrocell, Trace port interface unit); System Control Space including the Nested Vectored Interrupt Controller (NVIC) (Internal PPB); Fetch patch and breakpoint unit; Data watchpoint and trace unit; Instrumentation trace macrocell.

External RAM, External Device and PPB

External RAM region	Primarily used to store large data blocks or memory caches; off-chip memory, slower than the on-chip SRAM region.
External device region	Primarily used to map to external devices; off-chip devices, such as an SD card.
Private Peripheral Bus (PPB)	Provides access to internal and external processor resources.

Bit-band Operations

Bit-band operations allow a **single load/store operation to access a single bit in memory**, for example to change a single bit of one 32-bit data.

- **Normal operation without bit-band** (read-modify-write): read the value of the 32-bit data, modify a single bit of the 32-bit value (keeping other bits unchanged), write the value back to the address.
- **Bit-band operation**: directly write a single bit (0 or 1).

Bit-band Alias Addressing

The **SRAM and Peripheral regions** include bit-band and bit-band alias areas. Each bit of the data is **one-to-one mapped** to the bit-band alias address.

Region	1 MB bit-band	31 MB non-bit-band	32 MB bit-band alias
SRAM	0x20000000	0x20100000–0x21FFFFFF	0x22000000–0x23FFFFFF
Peripheral	0x40000000	0x40100000–0x41FFFFFF	0x42000000–0x43FFFFFF

- Each bit has an address that is a **multiple of 4**: 1 byte = 8 bits, so it occupies $4 \times 8 = 32$ addresses in memory space.
- Address translation: $\text{bit_word_addr} = \text{bit_band_base} + (\text{byte_offset} * 32) + (\text{bit_number} * 4)$. The programmer (or HAL/driver code) must do the address translation using the formula.

Real data address	Bit-band alias address
0x20000000	0x22000000
0x20000004	0x22000080
0x20000008	0x22000100
0x20000000 (bit [3])	0x2200000C

Example and Benefits

To set bit [3] in the word at address 0x20000000:

- without bit-band, bitmasking must be done manually and one risks **race conditions** if multiple masters/interrupts access the same word: `LDR R1, =0x20000000 ; LDR R0, [R1] ; ORR.W R0, #0x8 ; STR R0, [R1]`.
- with bit-band (using the alias), special hardware guarantees **atomic single-bit access**: `LDR R1, =0x2200000C ; MOV R0, #1 ; STR R0, [R1]`.
- **Benefits**: faster bit operations and fewer instructions; atomic operation, avoids hazards.
- **Hazard example**: if an interrupt is triggered and served during the read-modify-write operation and the interrupt service routine modifies the same data, a data conflict occurs: the bit modified by the ISR is overwritten by the main program when it writes its value back.

Cortex-M4 Endianness

- **Endian** refers to the order of bytes stored in memory.
 - **Little endian**: lowest byte of a word-size data is stored in bit 0 to bit 7.
 - **Big endian**: lowest byte of a word-size data is stored in bit 24 to bit 31.
- Cortex-M4 supports **both** little endian and big endian; however, endianness only exists at the hardware level.

Address	31:24	23:16	15:8	7:0
0x00000000 (little)	Byte3	Byte2	Byte1	Byte0
0x00000000 (big)	Byte0	Byte1	Byte2	Byte3

Arm and Thumb Instruction Set

Arm Instructions

- The early Arm instruction set is a **32-bit instruction set**, called the **Arm instructions**: powerful and good performance, but with larger program memory compared to 8-bit and 16-bit processors and larger power consumption, as more information needs to be moved in and out of memory.
- Example: `ADDS Rd, Rd, #Constant`.

Thumb-1

- **Thumb-1** is a **16-bit instruction set** (a subset of the Arm instruction set functionality): each instruction has a corresponding 32-bit Arm instruction that performs the operation.
- Example: `ADD Rd, #Constant` encodes the same operation using fewer bits than the Arm `ADDS Rd, Rd, #Constant`.
- Better code size compared to a 32-bit RISC architecture, but it sometimes leads to an increased number of instructions, making Thumb slower to execute (fetching instructions is slower than executing instructions): **code size is reduced by 30%, but performance is also reduced by 20%**.

Mixing Arm and Thumb

- A **mix of Arm and Thumb-1** instruction sets benefits from both 32-bit Arm (high performance) and 16-bit Thumb-1 (high code density).
- A **multiplexer** is used to switch between the two states, Arm state (32-bit) and Thumb state (16-bit), which introduces a **switching overhead**.
- The **T bit** selects the state: 0 selects Arm, 1 selects Thumb. The processor and the compiler manage all this.

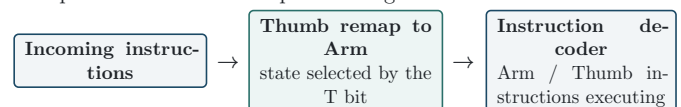


Fig. 8: A multiplexer switches between Arm state (T = 0, 32-bit) and Thumb state (T = 1, 16-bit); incoming instructions are remapped to Arm when needed and decoded before execution.

Thumb-2

- The **Thumb-2** instruction set consists of both **32-bit Thumb instructions** and the original **16-bit Thumb-1** instruction set.
 - Compared to the 32-bit Arm instruction set, **code size is reduced by 26%, with similar performance**.
- It covers almost all the functionality of the Arm instruction set.
 - Most Thumb-2 instructions are **unconditional**, whereas almost all Arm instructions can be **conditional**.